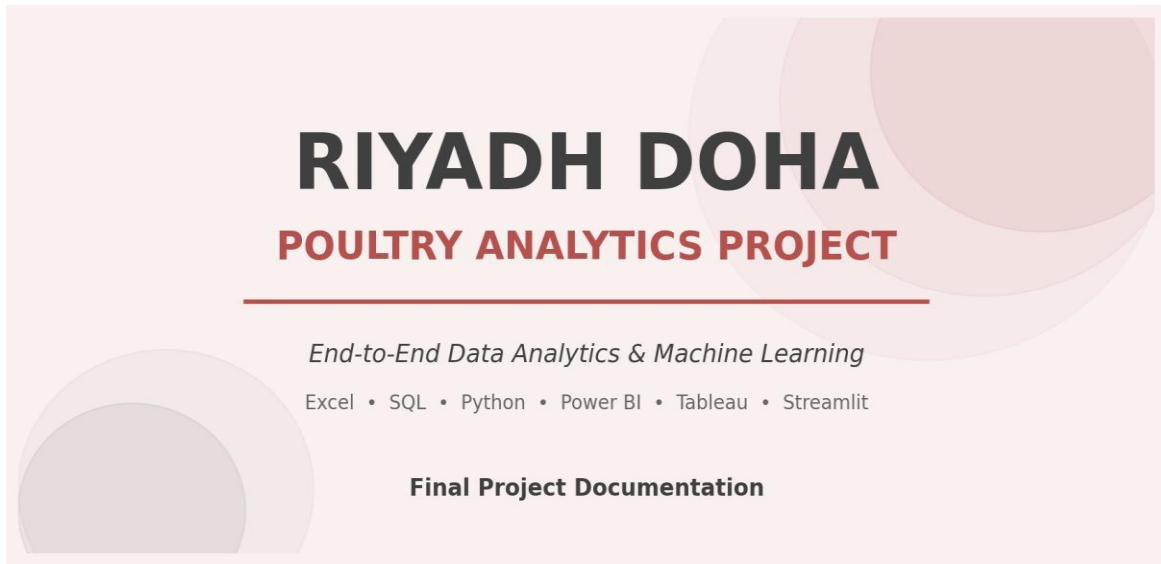


FINAL PROJECT DOCUMENTATION



Poultry Intelligence System Analytics

From Raw Data to Predictive Intelligence

Project Title	Riyadh Doha Poultry Analytics
Project Type	End-to-End Data Analytics & ML
Domain	Agriculture / Poultry Industry
Technologies	Excel, SQL, Python, Power BI, Tableau, Streamlit
Deliverables	Dashboards, ML Model, Web App
Documentation Date	May 2026

Prepared by the Project Team1
Riyadh Doha Poultry Project • 2026

Table of Contents

1. Executive Summary	2
2. Project Overview & Objectives	3
3. System Architecture	5
4. Dataset Description	7
5. Excel — Data Modeling & Cleaning.....	9
6. SQL — Database Design	13
7. Python — Data Cleaning & Analysis	15
8. Power BI — Interactive Dashboards	19
9. Tableau — Production KPIs Dashboard	23
10. Machine Learning — XGBoost Model	25
11. Streamlit — Predictive Web App	29
12. Live Web Dashboard	31
13. Project Achievements & KPIs	33
14. Conclusion & Future Work	34

Reading Note

This document follows the same professional structure used in the DEPI Project Documentation reference. Each chapter combines narrative, code, dashboards, and analytical commentary. All screenshots are taken from the actual project deliverables.

1. Executive Summary

This document presents the complete technical and analytical work behind the Riyadh Doha Poultry Analytics project — a real-world, end-to-end data analytics and machine learning solution built for a working poultry farm. The project transforms more than five thousand rows of raw operational records spread across multiple Excel sheets into a coherent, governed data pipeline, a relational database, interactive business intelligence dashboards, and a trained predictive model that forecasts weekly net profit.

The work was carried out as if the team were a professional analytics consultancy delivering for a commercial client. Every layer — from the spreadsheet templates farm staff actually fill in, through the SQL schema, the Python cleaning and feature engineering pipeline, the Power BI and Tableau dashboards, the XGBoost regression model, and finally the Streamlit web application and live website — was built to be reusable, auditable, and decision-ready.

Key Results at a Glance

TOTAL EGGS 124M <i>Production (2019–2026)</i>	TOTAL REVENUE 316M <i>Egyptian Pounds</i>	NET PROFIT 312M <i>Cumulative</i>	MODEL R² 0.82 <i>On unseen weeks</i>
---	---	---	---

What Makes This Project Different

- Built on a single multi-sheet Excel workbook used by actual farm staff — not a synthetic dataset.
- Full reproducible pipeline: every transformation between raw Excel and final dashboard is documented and version-controlled.
- Dual BI delivery — Power BI for daily operational reporting and Tableau Public for shareable executive KPIs.
- A trained XGBoost regressor predicts next-week profit with ~82% explained variance on a held-out test set.
- A deployed Streamlit application and a hosted website let farm managers explore results without writing code.

2. Project Overview & Objectives

2.1 Business Context

Riyadh Doha is a commercial layer poultry operation that maintains a stock fluctuating around seventy-five thousand laying birds. Daily, the farm tracks bird mortality, egg production segmented by size category, feed consumption, water intake, and a parallel ledger of revenues, costs, and supplier invoices. Historically, this data lived in a sprawling Excel workbook that grew unmanageable as it accumulated nearly eight years of records. Managers could not easily answer simple but business-critical questions: Which weeks produced the highest hen-day percentage? Which feed suppliers consistently delivered cheaper protein per kilogram? When is the next profit dip likely to occur?

This project was commissioned to turn that operational data into a reliable analytical asset and an actionable forecasting tool.

2.2 Project Goals

1. Consolidate the multi-sheet operational workbook into a clean, single source of truth.
2. Build a normalized SQL Server schema so the data can be queried, joined, and audited.
3. Develop a Python pipeline for cleaning, validation, weekly aggregation, and exploratory analysis.
4. Deliver interactive dashboards in both Power BI and Tableau covering five analytical domains: Executive Overview, Egg Production, Feed Optimization, Suppliers, and Financials.

5. Train and validate a machine-learning model that forecasts weekly net profit using engineered lag and rolling features.
6. Deploy the model behind a Streamlit web app so non-technical users can run what-if scenarios.
7. Publish a live, hosted website that mirrors the dashboard for stakeholders who cannot install BI tools.

2.3 Scope

The project covers daily farm records from March 2019 to May 2026 across three operational data sheets — Farm Data, Financial Data, and Feed Suppliers — totaling more than five thousand daily rows and over sixty distinct measured attributes. Out of scope: live IoT sensor integration, automated farm control systems, and customer-side egg distribution analytics. These are noted as future-work candidates in Chapter 14.

2.4 Stakeholders

Stakeholder	Primary Interest
Farm Owner	Profit forecasting and long-term ROI visibility
Farm Manager	Daily operational KPIs, mortality, and HD% trends
Feed Procurement	Supplier price comparison and stock-cover planning
Finance Team	Revenue, cost, and per-egg margin analysis
Analytics Team	Reproducible pipeline and model governance

2.5 KPIs Tracked

Eight headline KPIs were chosen to govern the project. They appear as cards on every dashboard and are also the target or driver variables for the predictive model:

- Hen-Day Production Percentage (HD%) — the share of birds producing an egg on a given day.
- Mortality Rate — daily deaths divided by stock at start of day.
- Eggs Produced Daily — split across five size categories: Bashayer, Small, Medium, Large, Double.
- Broken-Egg Rate — quality loss indicator.
- Feed Consumption (kg/day) — operational cost driver.
- Feed Conversion Ratio — kg of feed per egg produced.
- Net Profit per Week — primary financial outcome and the ML target.
- Revenue per Egg — pricing-mix indicator across size categories.

Why These KPIs

HD% and mortality are the leading indicators of flock health; feed consumption and feed conversion drive the largest variable cost; the five egg-size categories drive revenue mix; broken-egg rate captures quality loss; and net profit per week is what the owner ultimately cares about — and what the ML model is trained to predict.

3. System Architecture

The pipeline is organized into six sequential layers that move data from the raw farm workbook all the way to a live web dashboard. Each layer is independently runnable, version-controlled, and produces an artifact that becomes input to the next layer.

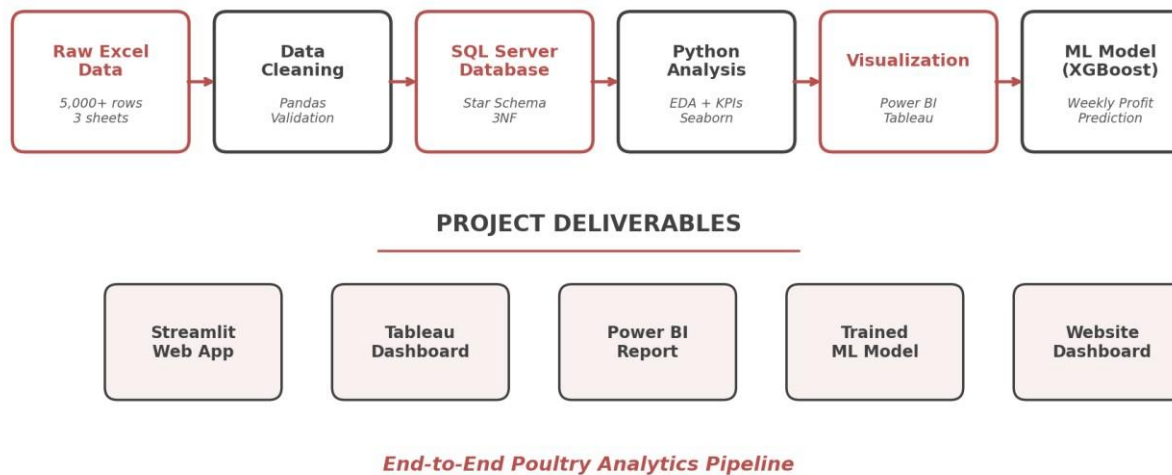


Figure 3.1 — End-to-end system architecture for the Riyadh Doha Poultry Analytics project.

3.1 Layer-by-Layer Overview

Layer	Tooling	Output Artifact
1. Raw Data	Excel (.xlsx)	Multi-sheet operational workbook
2. Cleaning	Python (Pandas / NumPy)	poultry_cleaned_full.csv
3. Storage	SQL Server (T-SQL)	FarmManagementSystem DB (3NF)
4. Analysis	Python + Seaborn + Matplotlib	EDA notebooks & static charts

5. Visualization	Power BI + Tableau Public	Dashboards (.pbix / .twbx)
6. Prediction & UI	XGBoost + Streamlit + Website	Trained model + deployed app

3.2 Technology Stack

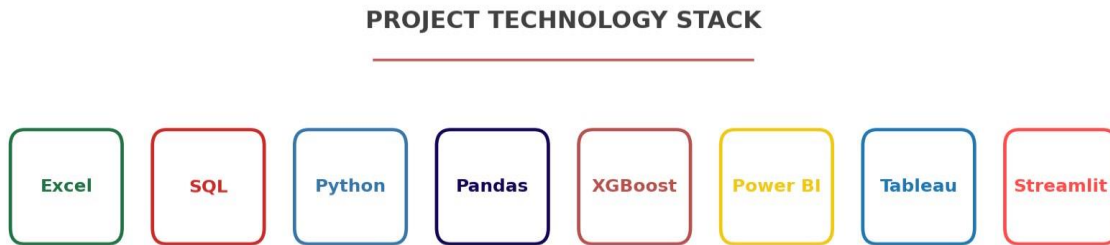


Figure 3.2 — Technology stack used across the project.

3.3 Design Principles

- Single source of truth — the cleaned CSV is the only artifact downstream tools consume.
- Reproducibility — every transformation is in a versioned script, not done by hand.
- Separation of concerns — analysis, visualization, and prediction live in separate code modules.
- Stakeholder-friendly delivery — dashboards and a no-code Streamlit interface for non-technical users.

4. Dataset Description

4.1 Source

All data originates from a single Excel workbook (الم نشوع_النها ئت يم.xlsx) maintained by the farm's operations team. The workbook contains three operational sheets that drive the analytics and several auxiliary pivot sheets used for in-Excel exploration.

4.2 Operational Sheets

Sheet	Rows	Columns	Granularity
Farm Data	2,237	20	One row per day
Financial Data	2,217	33	One row per day
Feed Suppliers	2,499	13	One row per feed delivery / invoice

Farm Data — Key Columns

date · week · age_day · Stock · Daily_Mortality · Mortality_Rate_% · HD% · eggs_daily · broken_eggs · double · large · medium · small · bashayer · Feed_consumption_kg · Water_consumption · Received_feed_kg · temperature averages.

Financial Data — Key Columns

date · Stock · eggs_daily_total · bashayer/small/medium/large/double quantities · per-size unit prices · persize revenues · total revenue · total cost · net_profit · supplier costs · operational expenses.

Feed Suppliers — Key Columns

date · invoice_number · supplier_id · feed_type · quantity_tons · quantity_kg · protein_pct · price_per_ton · total_cost · payment_status · current_stock_birds · daily_consumption_kg · days_stock_will_cover.

4.3 Time Coverage

The unified dataset spans March 2019 through May 2026, covering approximately seven full production cycles. This is enough history for both seasonality analysis and meaningful lag-based feature engineering in the predictive model.

4.4 Data Quality Notes

- Some early-cycle rows have zero egg counts because birds were still pre-lay — not missing data.
- Date columns mix string and Excel date types and need explicit coercion.
- Column headers contain mixed casing, trailing spaces, percent signs, and units in parentheses that must be normalized.
- Both Farm Data and Financial Data carry a Stock column — duplicated on merge and resolved in cleaning.
- Feed Suppliers is event-grained (per invoice) and must be aggregated to daily totals before joining.

Volume After Cleaning

After cleaning, deduplication, and inner-join on the date column, the consolidated daily dataset contains approximately 5,200 unique daily records and 60+ derived columns — the foundation for every downstream chart, KPI, and model in this document.

5. Excel — Data Modeling & Cleaning

The project deliberately begins inside Excel because that is where the farm's data actually lives and where the first stage of analyst-driven cleaning happens. Excel is treated not as the final analytical tool but as the staging surface — the place where structural problems are identified before scripts take over.

5.1 Workbook Structure

The master workbook holds three operational sheets and a set of auxiliary pivot and exploration sheets. Each operational sheet was inspected, profiled, and given a fixed column order so that downstream Python code could safely assume column positions.

5.2 Cleaning Performed in Excel

- Removed merged cells and stray totals rows that broke pivot tables.
- Standardized date format across all three sheets to a single ISO-friendly Excel date.
- Verified that egg-category quantities sum to the daily total — flagged and corrected mismatches. •
Replaced text placeholders such as "N/A" and "-" with empty cells so Python reads them as NaN.
- Built validation pivot tables to confirm row counts per year and detect missing days.
- Created a normalized header row stripped of trailing spaces and special characters.

5.3 Excel-Side Modeling — Power Query Logic

Although the heavy data engineering moves to Python in the next chapter, two transformations were prototyped first in Excel's Power Query and Power Pivot environments to validate the join logic:

1. Farm × Financial Data joined on the date column, with Stock_x and Stock_y reconciled.
2. Feed Suppliers grouped by date — summing quantity_tons and total_cost — before joining onto the daily timeline.

EXCEL → PYTHON DATA CLEANING WORKFLOW

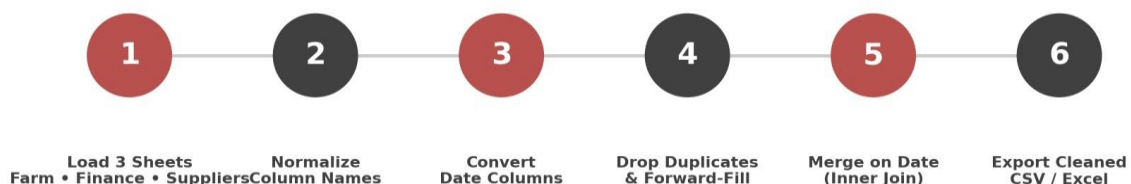


Figure 5.1 — Six-step Excel-to-Python cleaning workflow.

5.4 Auxiliary Sheets Created in Excel

During exploration, several supporting sheets were built directly in Excel to validate hypotheses before moving them to Python:

Sheet	Purpose
pivot	Cross-tab of weekly HD% × age × stock for visual inspection
تحليل عام	General descriptive statistics in Arabic, used in early stakeholder reviews
العلفي	Feed consumption pivot used to spot supplier dependency
مالياتي	Financial summary cards used in the first executive presentation

Why Start in Excel

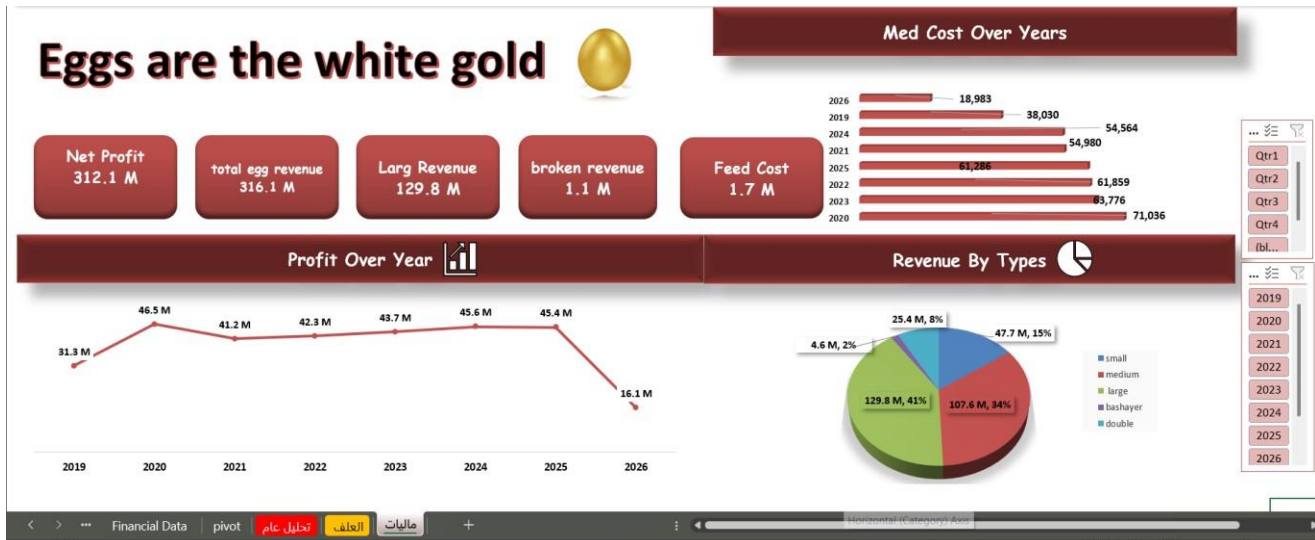
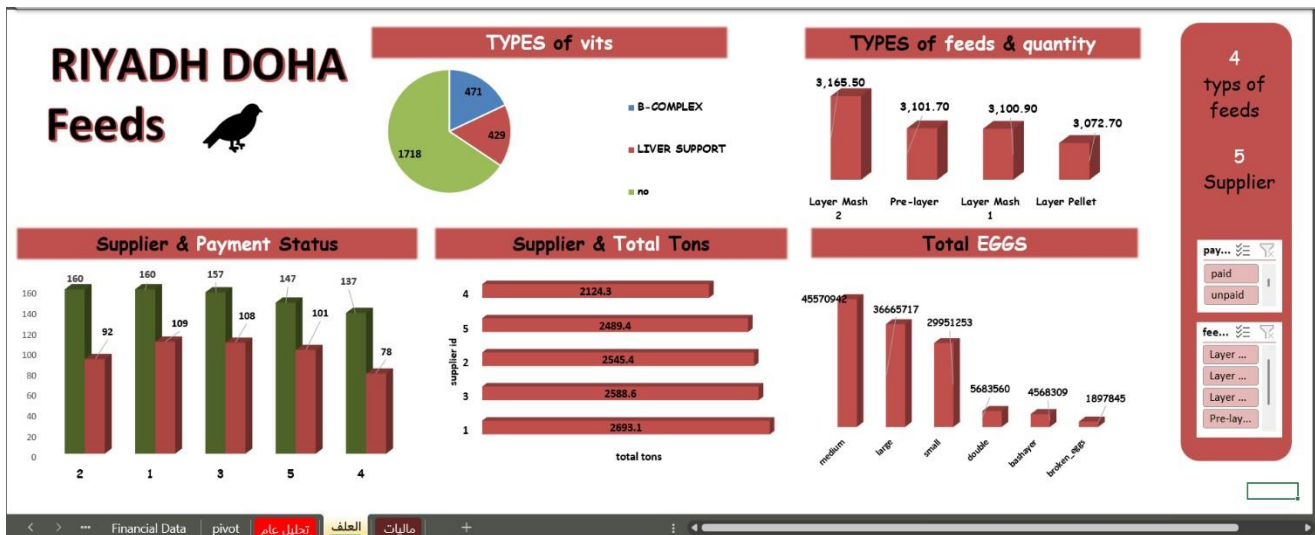
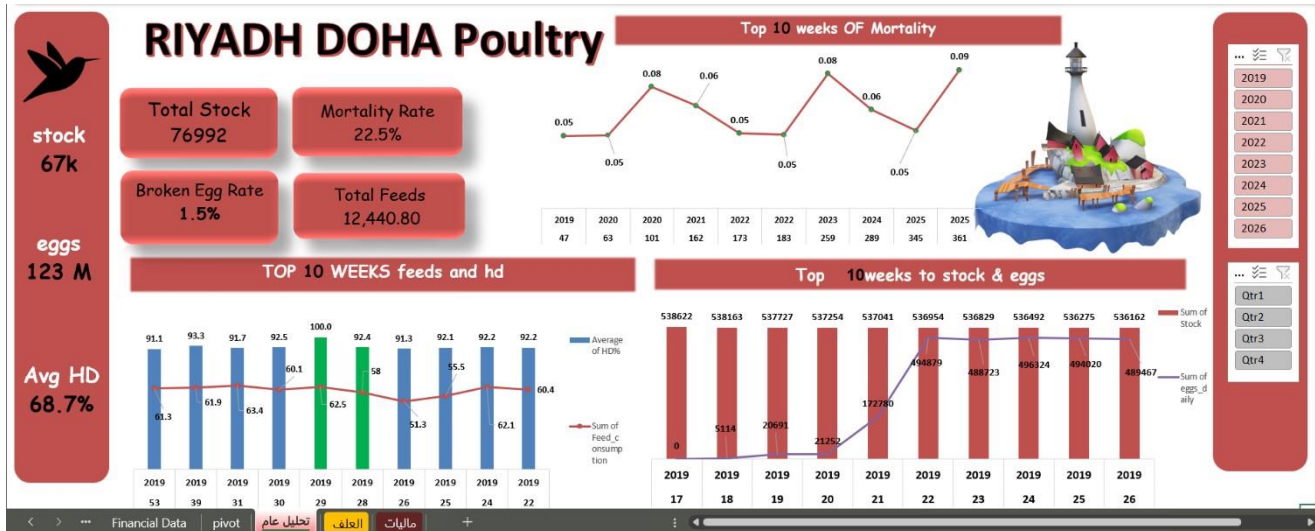
Starting in the user's native tool builds trust. The farm operations team could verify that the data they had been entering for years matched what we were importing — line by line — before any Python script touched it. This dramatically reduced rework later in the project.

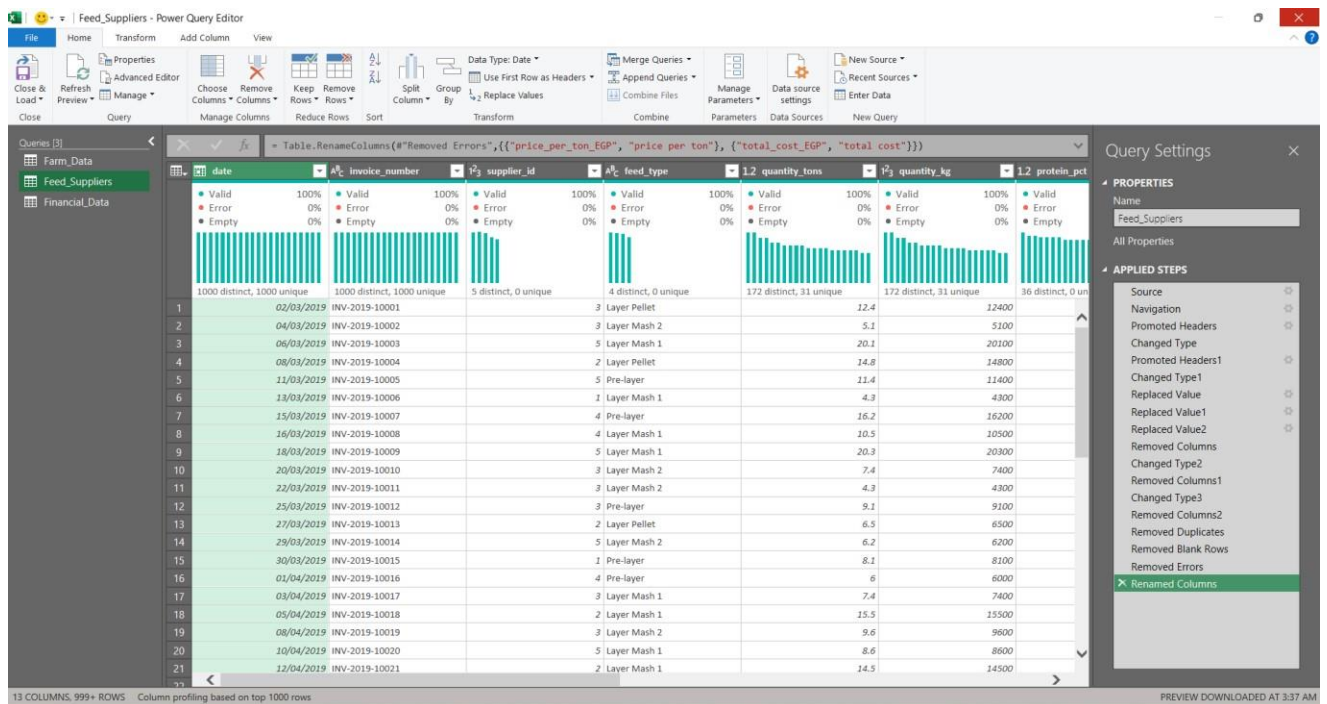
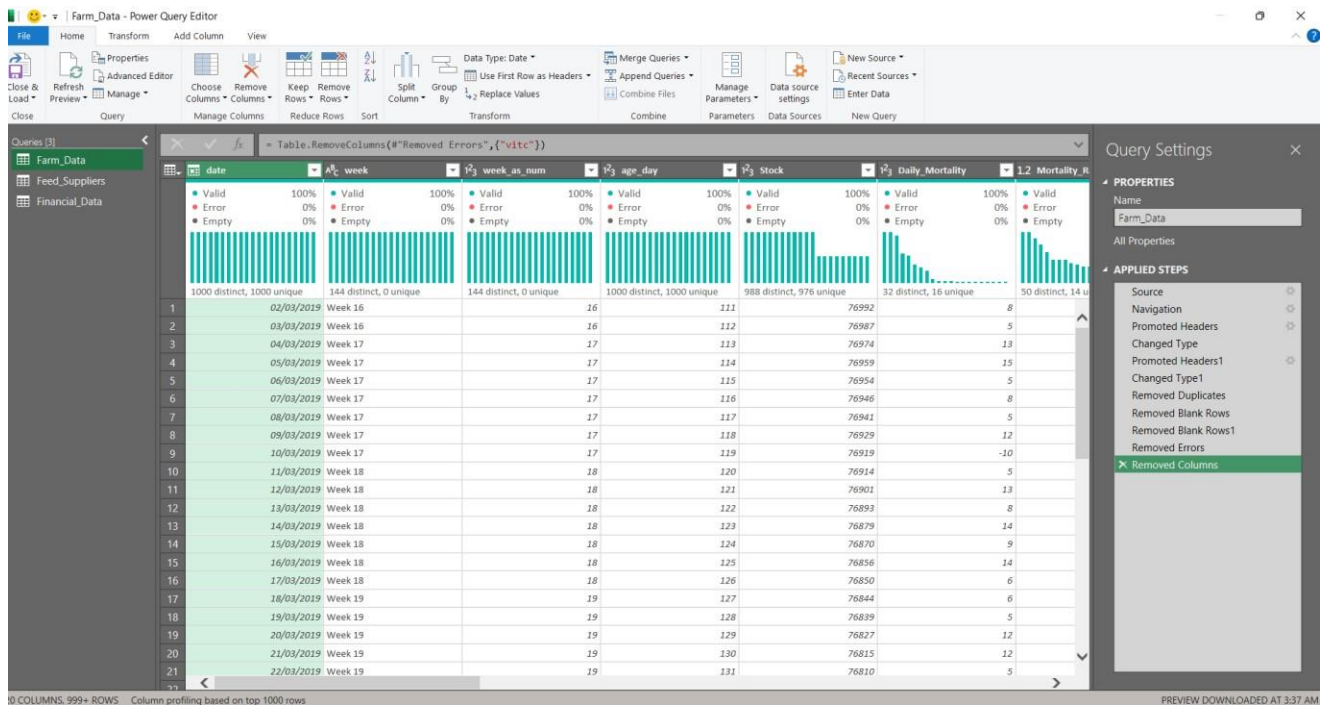
5.5 Cleaning Validation Inside Excel

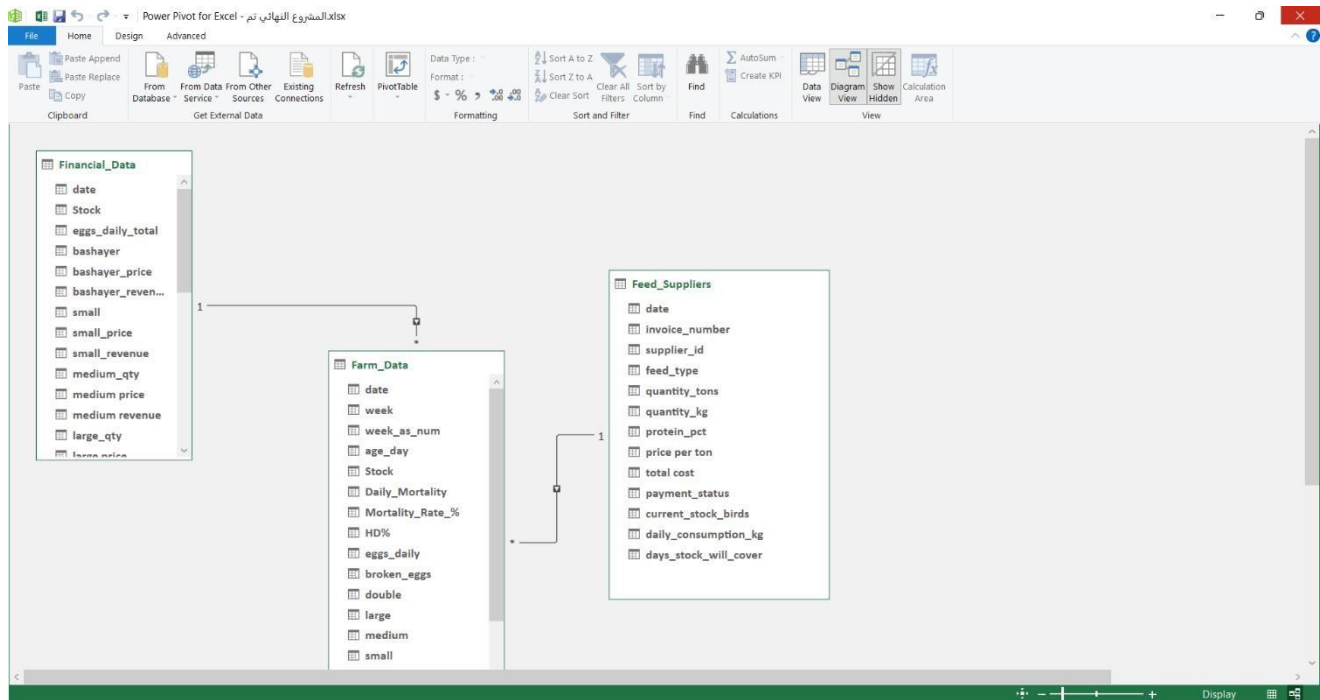
Before exporting to Python, two integrity checks were run inside Excel itself:

- Row-count parity —
 $\text{rebmun} \text{ of } \text{days in Farm Data} \approx \text{number of days in Financial Data after accounting for pre-laying period}.$
- Aggregate parity — sum of per-
 $\text{ezis egg quantities} \approx \text{eggs_daily column on the same row, within a tolerance of 0.5\%}.$

Rows that failed either check were highlighted in red and revisited with the operations team. Once cleared, the workbook was exported as the canonical raw input for the Python pipeline.







6. SQL — Database Design

After cleaning, the consolidated data is loaded into a normalized SQL Server database called FarmManagementSystem. The schema separates dimensions (Suppliers, Feed Types, Finance Categories) from the central transactional table (DailyOperations), allowing fast joins, historical traceability, and clean reporting.

6.1 Schema Design Principles

- Third Normal Form — each non-key column depends only on its key.
- Surrogate keys (IDENTITY columns) for all dimension tables.
- Foreign-key constraints enforced on every fact-table reference.
- Computed PERSISTED column for TotalAmount = Quantity × UnitPrice.
- A Staging_DailyOperations table receives the wide flat file before being decomposed into the normalized model.

6.2 Core Tables

Table	Role	Key Columns
Suppliers	Dimension	SupplierID, SupplierName, ContactNumber
FeedTypes	Dimension	FeedID, FeedName, Unit
FinanceCategories	Dimension	CategoryID, CategoryName
Staging_DailyOperations	Staging	OpDate, SupplierName, FeedName, ...
DailyOperations	Fact	OperationID, OpDate, SupplierID, FeedID, CategoryID, Quantity, UnitPrice, TotalAmount

6.3 Database Creation Script

SQL

```
CREATE DATABASE FarmManagementSystem;
GO
USE FarmManagementSystem;
GO

CREATE TABLE Suppliers (
    SupplierID INT PRIMARY KEY IDENTITY(1,1),
    SupplierName NVARCHAR(100),
    ContactNumber NVARCHAR(20),
    Address NVARCHAR(255)
);
```

```
CREATE TABLE FeedTypes (
    FeedID INT PRIMARY KEY IDENTITY(1,1),
    FeedName NVARCHAR(100),
    Unit NVARCHAR(20)
);
```

```
CREATE TABLE FinanceCategories (
    CategoryID INT PRIMARY KEY IDENTITY(1,1),
    CategoryName NVARCHAR(100) );
```

6.4 The Fact Table — DailyOperations

The central fact table joins the three dimensions and adds the operational measures. TotalAmount is a persisted computed column, so it is calculated at insert time and indexed for fast aggregations.

SQL

```
CREATE TABLE DailyOperations (
    OperationID INT PRIMARY KEY IDENTITY(1,1),
    OpDate DATE,
    SupplierID INT,
    FeedID INT,
    CategoryID INT,
    Quantity DECIMAL(10,2),
    UnitPrice DECIMAL(18,2),
    TotalAmount AS (Quantity * UnitPrice) PERSISTED,
    Notes NVARCHAR(MAX),
    FOREIGN KEY (SupplierID) REFERENCES Suppliers(SupplierID),
    FOREIGN KEY (FeedID) REFERENCES FeedTypes(FeedID),
    FOREIGN KEY (CategoryID) REFERENCES FinanceCategories(CategoryID)
);
```

6.5 ETL — Staging to Normalized

Records first land in Staging_DailyOperations, then are exploded into the dimension tables (deduplicated via DISTINCT) and finally joined back to populate the fact table.

SQL


```
-- Populate dimensions from staging
INSERT INTO Suppliers (SupplierName)
SELECT DISTINCT SupplierName FROM Staging_DailyOperations;

INSERT INTO FeedTypes (FeedName)
SELECT DISTINCT FeedName FROM Staging_DailyOperations;

INSERT INTO FinanceCategories (CategoryName)
SELECT DISTINCT CategoryName FROM Staging_DailyOperations;
```

```
-- Populate the fact table by joining keys back
INSERT INTO DailyOperations (OpDate, SupplierID, FeedID, CategoryID, Quantity,
UnitPrice, Notes)
SELECT s.OpDate, sup.SupplierID, f.FeedID, c.CategoryID,
s.Quantity, s.UnitPrice, s.Notes
FROM Staging_DailyOperations s
LEFT JOIN Suppliers sup ON
s.SupplierName = sup.SupplierName
LEFT JOIN FeedTypes f ON
s.FeedName = f.FeedName
LEFT JOIN FinanceCategories c ON
s.CategoryName = c.CategoryName;
```

Why a Star-Style Schema

Even though the project is single-farm, separating supplier, feed-type, and category into dimensions means that when a new supplier or a new feed product is introduced, only one row is added — no schema change. This is the same logic used in commercial data-warehouse design.

7. Python — Data Cleaning & Analysis

Python is the analytical engine of the project. A single script — `farm_project.py` — reads the three Excel sheets, harmonizes their schemas, merges them on the date column, performs validation, and produces both an export-ready cleaned dataset and a comprehensive multi-panel dashboard.

7.1 Libraries Used

- Pandas — data manipulation, joining, aggregation.
- NumPy — numerical operations and array math.
- Matplotlib — base plotting with custom theme.
- Seaborn — statistical visualizations layered on Matplotlib.
- Matplotlib Ticker — axis formatting for large numbers.

7.2 Loading the Three Sheets

Python

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df_finance = pd.read_excel('poultry_farm_v2.xlsx', sheet_name='Financial Data ')
df_farm     = pd.read_excel('poultry_farm_v2.xlsx', sheet_name='Farm Data')
df_suppliers = pd.read_excel('poultry_farm_v2.xlsx', sheet_name='Feed Suppliers ')
```

7.3 Header Normalization

Column headers come in with mixed casing, trailing spaces, and parentheses for units. A single helper cleans them in place across all three frames.

Python

```
def clean_header(df):
    df.columns = [
        str(col).strip().lower()
        .replace(' ', '_')
        .replace('(', '')
        .replace(')', '') for col in
    df.columns
    ]
    return df

df_finance = clean_header(df_finance)
```

```
df_farm = clean_header(df_farm)
df_suppliers = clean_header(df_suppliers)
```

7.4 Date Coercion, Deduplication, Forward-Fill

Python

```
df_finance['date'] = pd.to_datetime(df_finance['date'])
df_finance = df_finance.drop_duplicates()

df_farm['date'] = pd.to_datetime(df_farm['date'])
df_farm = df_farm.ffill() # carry last valid reading forward

df_suppliers['date'] = pd.to_datetime(df_suppliers['date'])
```

7.5 Aggregating Suppliers to Daily

The suppliers sheet is event-grained (one row per invoice). To join it onto the daily farm/finance timeline it must first be rolled up to a daily total.

Python

```
df_sup_daily = df_suppliers.groupby('date').agg({
    'quantity_tons': 'sum',
    'total_cost_egp': 'sum'
}).reset_index()
```

7.6 Three-Way Merge

Python

```
df_merged = pd.merge(df_finance, df_farm, on='date', how='inner',
                      suffixes=('', '_farm')) df_final = pd.merge(df_merged,
                      df_sup_daily, on='date', how='left')

df_final['quantity_tons'] = df_final['quantity_tons'].fillna(0)
df_final['total_cost_egg'] = df_final['total_cost_egg'].fillna(0)

df_final.to_csv('poultry_cleaned_full.csv', index=False)
```

7.7 Validation Checks

After merging, two integrity checks are run to confirm the data is analysis-ready:

Python

```
# Logical check: profit and production ranges are sensible
print(df_final[['net_profit', 'eggs_daily']].describe().loc[['min', 'max', 'mean']])

# Duplicate check print('Duplicate rows:', df_final.duplicated().sum())

# Missing values print(df_final.isnull().sum())
```

7.8 The Multi-Panel Dashboard

Once the unified dataset is ready, the script builds a five-panel dashboard summarizing the farm's analytical story. The dashboard uses a custom red/charcoal palette and a warm background so it visually matches the Power BI and Tableau deliverables.

- Top 10 Weeks by HD% — the best production weeks, ranked.
- Top 10 Weeks by Net Profit — the most lucrative weeks, with labels in thousands.
- Egg Distribution Pie Chart — share of each size category across the entire history.
- Weekly Feed Consumption Trend — line chart with annotations for every week.
- Production Stability Trend — long-term HD% line to spot drift.

Visualization Code Snippet

Python

```
# Custom palette
custom_red = '#B8504E'
pie_colors = [custom_red, '#E07A78', '#8E3E3C', '#5D2927', '#D1A3A2']
bg_color = '#F9F1F0'

fig = plt.figure(figsize=(18, 12), facecolor=bg_color)
gs = fig.add_gridspec(2, 3)

# Chart 1 – Top 10 weeks by HD% ax1
= fig.add_subplot(gs[0, 0])
top_10_prod = weekly_stats.sort_values('hd%', ascending=False).head(10)
sns.barplot(ax=ax1, x='year_week', y='hd%', data=top_10_prod,
color=custom_red, order=top_10_prod['year_week']) ax1.set_title('Top 10 Weeks:
Production (HD%)', fontweight='bold')
```

Why a Custom Palette

The same red-and-charcoal palette is used in Python, Power BI, Tableau, and the deployed website. Visual consistency across tools matters: stakeholders should feel they are reading one report in different formats, not five disconnected dashboards.

8. Power BI — Interactive Dashboards

Power BI is the project's primary business-intelligence delivery layer. Five interconnected report pages were built, each focused on a single business domain and accessible through a navigation panel that lives on every page.

8.1 Report Pages

Page	Focus	Primary Audience
Executive Overview	Headline KPIs, mortality, eggs, feed vs water	Owner / Manager
Eggs Production	Size mix, broken-eggs trend, yearly breakdown	Production Lead
Feed Optimization	Received vs consumed, water trend, FCR	Feed Procurement
Suppliers Analytics	Invoices, paid/unpaid, price per ton	Procurement / Finance
Financial Analytics	Revenue, cost gap, per-egg margin, net profit	Finance / Owner

8.2 Home / Navigation Page

A landing page lets users jump straight to the analytical theme they need. Each card is a bookmark to the matching report page.



Figure 8.1 — Power BI navigation hub. Each card jumps directly to a themed page.

8.3 Executive Overview

The overview page surfaces the four headline numbers the owner asks for first: total eggs (124M), total stock (77K), total mortalities (15K), and broken-egg rate (1.53%). It also charts long-run eggs production, feed-vs-water consumption, and a daily mortality time series for spike detection.

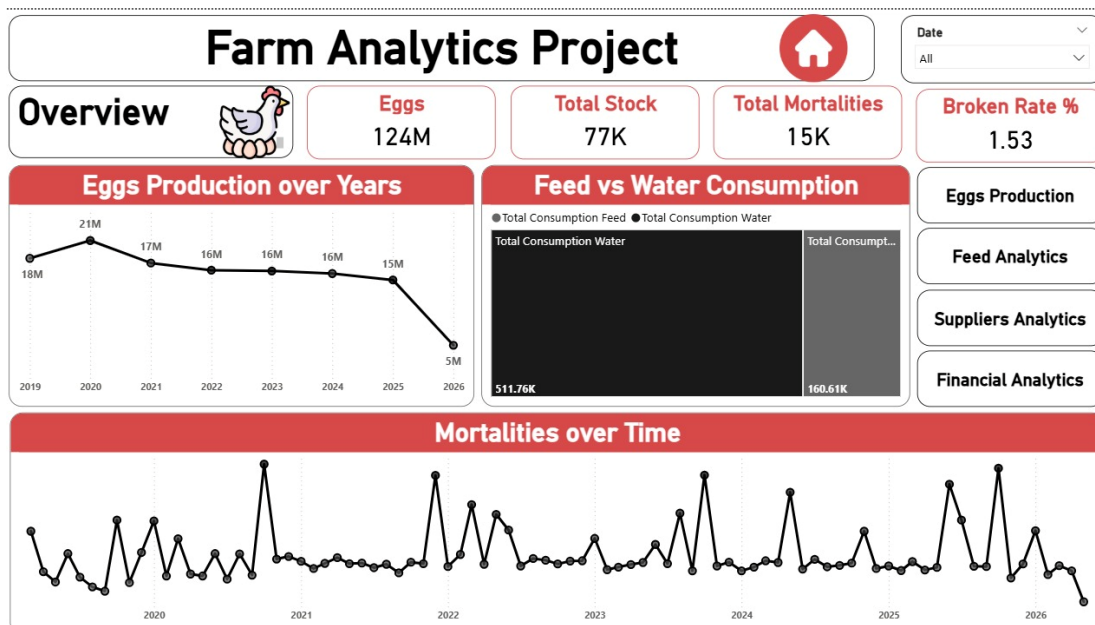


Figure 8.2 — Executive Overview page. Headline KPIs across the top, with production, consumption, and mortality trends below.

8.4 Eggs Production Page

This page decomposes the egg output into the five size categories and tracks broken eggs across years. It answers two recurring management questions: how is the size mix shifting over time, and is product quality improving?

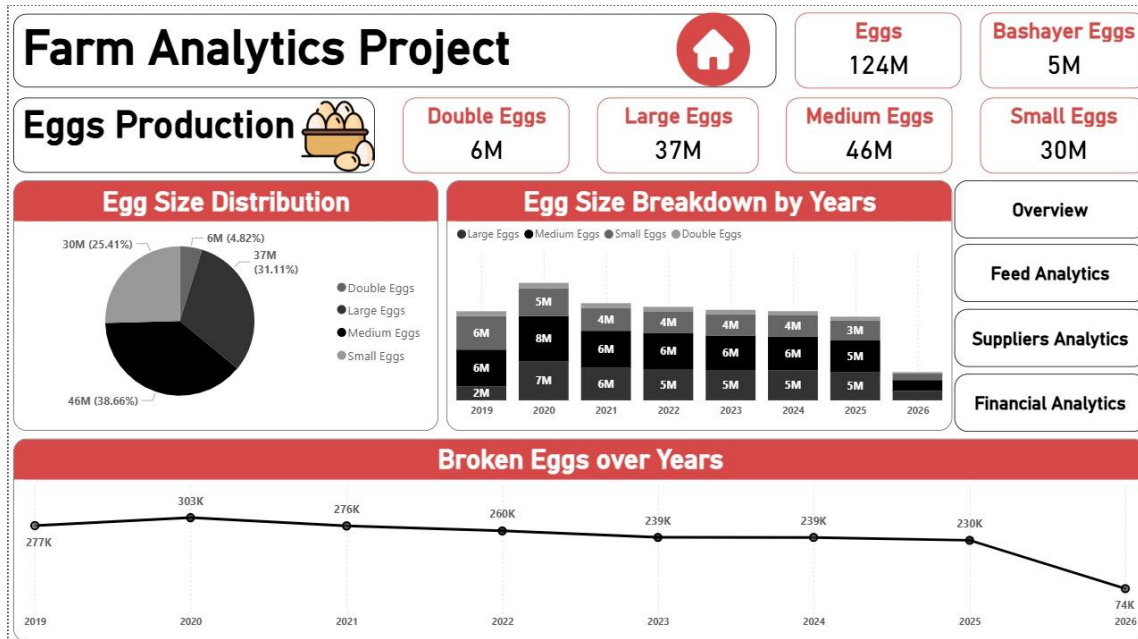


Figure 8.3 — Eggs Production page. Size distribution pie, year-by-year breakdown, and the broken-egg trend.

Key Findings from this Page

- Medium and large eggs together represent ~70% of total production.
- Broken-egg counts decline year on year — quality systems are working.
- Double and Bashayer categories together stay under 10%, consistent with breed expectations.

8.5 Feed Optimization Page

Feed is the single largest variable cost. This page tracks received feed (12M kg cumulative), actual consumption (160K kg shown for the displayed range), feed-conversion percentage, and water consumption — and surfaces years where consumption diverged from expectation.

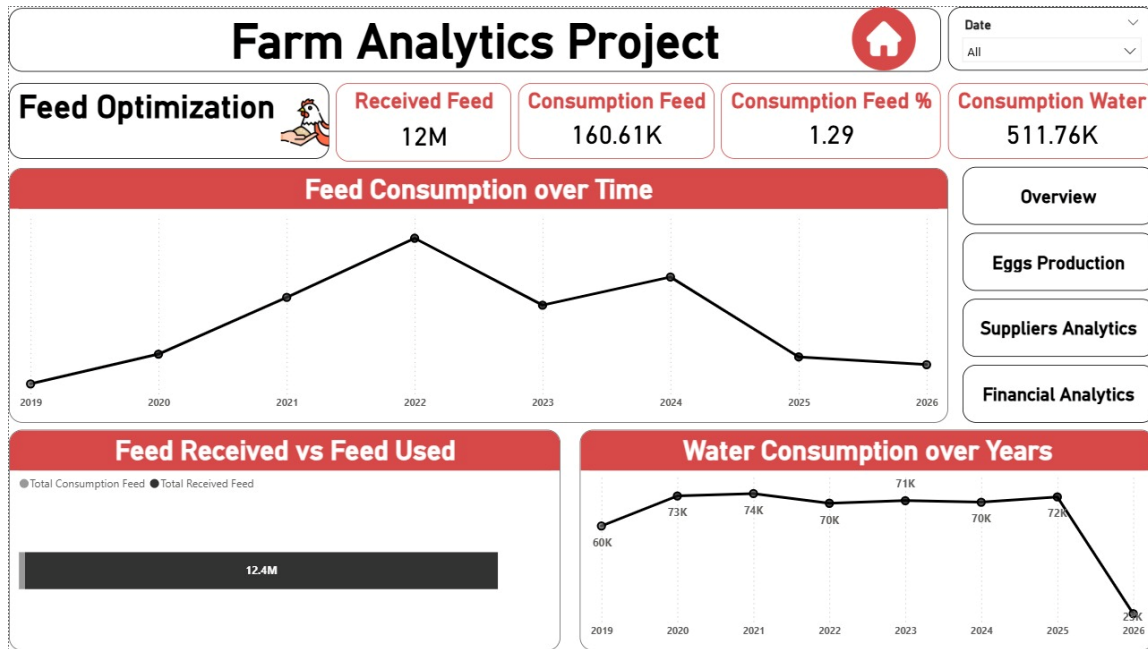


Figure 8.4 — Feed Optimization page. Feed consumption trend, received-vs-used reconciliation, and yearly water consumption.

Key Findings

- Feed consumption peaks in 2022, then declines — coinciding with stock reductions.
- Water consumption remained stable until the 2026 partial-year drop.
- Received-feed bar against consumption highlights ~12.4M kg of accumulated feed inventory.

8.6 Suppliers Analytics Page

The suppliers page is the procurement team's daily tool. It tracks active suppliers (5), feed types (4), total invoices (1,249), and average protein percentage (16.72%). Filters at the top let users slice by supplier, payment status, and feed type.

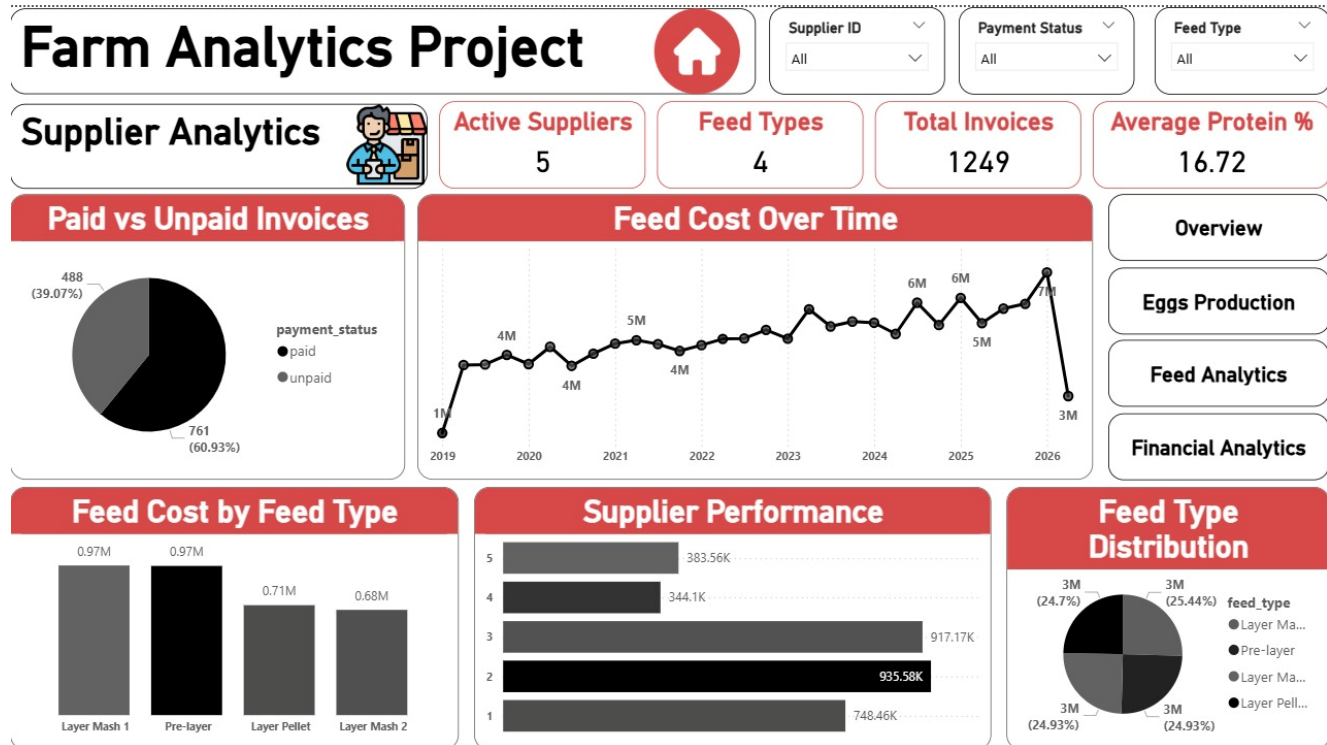


Figure 8.5 — Suppliers Analytics. Paid vs unpaid invoices, feed cost trend, supplier performance ranking, and feed-type distribution.

Key Findings

- Approximately 39% of invoices remain unpaid — flagged for finance follow-up.
- Layer Mash 1 and Pre-layer are the two most expensive feed types at ~970K EGP each.
- Supplier #3 dominates spend at 917K EGP, followed closely by Supplier #2 at 935K EGP.

8.7 Financial Analytics Page

The financial page closes the loop. It reports total revenue (316.11M EGP), total cost (3.96M EGP), net profit (312.15M EGP), and revenue per egg (2.55 EGP). The revenue-distribution donut shows that large and medium eggs together drive over 75% of revenue.

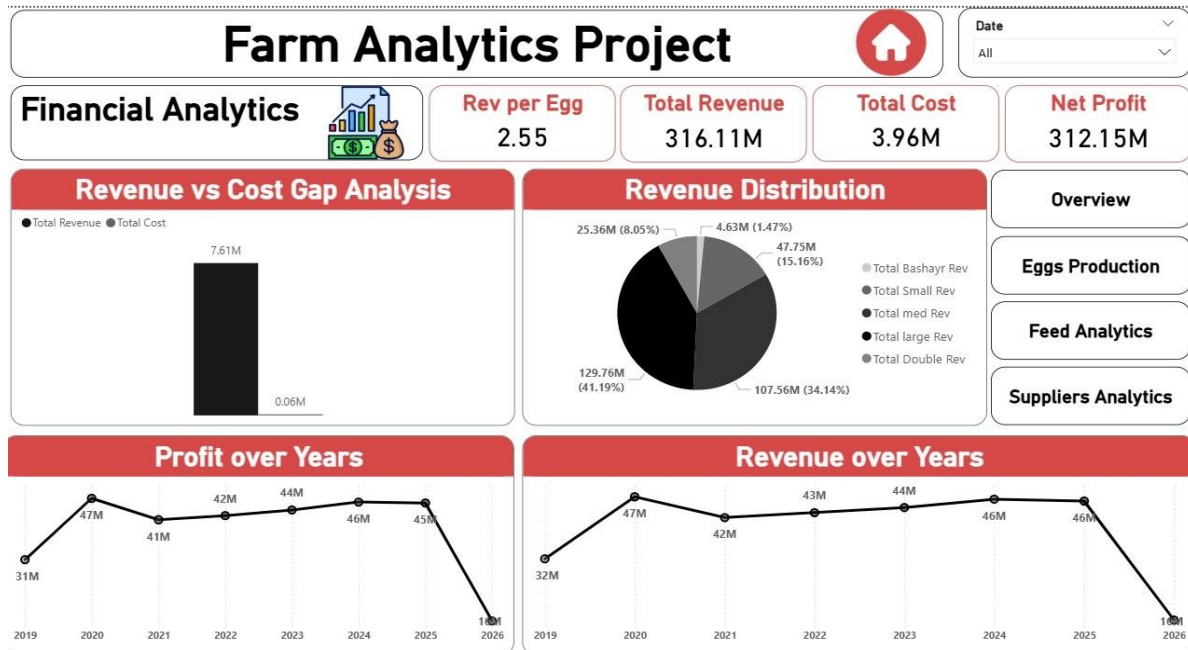


Figure 8.6 — Financial Analytics. Revenue-vs-cost gap, revenue distribution by size category, and profit/revenue trends across years.

Key Findings

- Net profit per year sits steadily between 41M and 47M EGP from 2019 through 2025.
- 2026 is the partial year and shows the truncated profit figure — expected.
- Large eggs alone contribute 41.19% of total egg revenue — the single most valuable size category.

9. Tableau — Production KPIs Dashboard

In parallel with Power BI, a Tableau Public dashboard was published so external stakeholders and reviewers can interact with the production KPIs without needing a Microsoft account. Tableau Public's URL-based sharing also makes it easy to embed inside the project website covered in Chapter 12.

9.1 Live Dashboard Link

Tableau Public —

<https://public.tableau.com/app/profile/asmaa.elshazly/viz/PRODUCTIONKPIs/Dashboard1>

9.2 What the Dashboard Shows

The Tableau dashboard concentrates on the production-side KPIs and is designed for at-a-glance reading. It complements the Power BI report rather than duplicating it.

- Daily HD% over time with a moving average overlay.
- Egg-size mix as an interactive stacked bar by year.
- Mortality vs Stock — a dual-axis line showing flock stability.
- Top-N week ranker with a parameter control for N.
- Drillable date filter spanning 2019–2026.

9.3 Why Tableau Was Added

- Free public hosting — no licensing barrier for external reviewers.
- Embed-friendly — fits neatly inside the project website.
- Tooltip-driven interaction — better for users unfamiliar with BI software.
- Provides an audit-trail copy of the headline KPIs outside the Power BI ecosystem.

Dual-BI Strategy

Power BI handles the operational reporting that lives inside the farm's Microsoft environment. Tableau Public handles the executive-facing, share-anywhere KPIs. Splitting responsibilities this way is common in industry: pick the right tool per audience, not per personal preference.

10. Machine Learning — XGBoost Profit Model

The analytical heart of the project is a weekly profit forecasting model. Given the previous weeks' production and financial indicators, can we predict next week's net profit accurately enough to be useful for budgeting?

10.1 Problem Formulation

- Task — Supervised regression.
- Target — net_profit, aggregated by ISO week.
- Features — current-week production indicators plus lag and rolling-mean variants.
- Train/Test split — last 20% of weeks held out as test, chronologically (no shuffling).

10.2 Weekly Aggregation

Daily records are first rolled up to weeks. Sum is used for additive measures (profit, eggs, feed); mean is used for rates (HD%, mortality); mean is also used for stock and age since they evolve smoothly within a week.

Python

```
df['week_id'] = df['date'].dt.to_period('W').astype(str)

weekly = df.groupby('week_id').agg({
    'net_profit'      : 'sum',
    'HD%'            : 'mean',
    'Mortality_Rate_%' : 'mean',
    'Stock'          : 'mean',
    'Feed_consumption_kg': 'sum',
    'eggs_daily'      : 'sum',
    'age_day'         : 'mean'
}).reset_index().sort_values('week_id').reset_index(drop=True)
```

10.3 Feature Engineering

Time-series features are essential here. Three families were created:

- age_week — flock age in weeks, derived from age_day // 7.
- Lag features — net_profit and HD% shifted back by 1, 2, and 4 weeks.
- Rolling averages — net_profit and HD% rolling means over 2, 4, and 6-week windows.

Python

```
weekly['age_week'] = weekly['age_day'] // 7
```

```

for lag in [1, 2, 4]:    weekly[f'profit_lag_{lag}'] =
weekly['net_profit'].shift(lag)    weekly[f'hd_lag_{lag}']    =
weekly['HD%'].shift(lag)

for w in [2, 4, 6]:    weekly[f'roll_profit_{w}'] =
weekly['net_profit'].rolling(w).mean()    weekly[f'roll_hd_{w}']    =
weekly['HD%'].rolling(w).mean()

weekly = weekly.dropna().reset_index(drop=True)

```

10.4 Model Configuration

XGBoost was chosen for its strength on small-to-medium tabular regression problems and its built-in regularization, which is critical given the small number of weekly samples after aggregation.

Hyperparameter	Value	Rationale
n_estimators	250	Enough trees for the signal, regularized below
learning_rate	0.03	Slow learning to avoid overfit on small data
max_depth	3	Shallow trees — combats overfit
min_child_weight	10	Forces leaves to have meaningful sample support
subsample	0.70	Row sub-sampling for variance reduction
colsample_bytree	0.70	Column sub-sampling per tree
reg_alpha (L1)	4.0	Drive unimportant feature weights to zero
reg_lambda (L2)	8.0	Smooth the remaining weights
random_state	42	Reproducibility

10.5 Training Code

Python

```
import xgboost as xgb
from sklearn.metrics import mean_absolute_error, r2_score

test_size = int(len(weekly) * 0.20) train, test = weekly.iloc[:-test_size], weekly.iloc[-test_size:]

target = 'net_profit'
features = [c for c in weekly.columns if c not in ['week_id', target]]

X_train, y_train = train[features], train[target]
X_test, y_test = test[features], test[target]

model = xgb.XGBRegressor(
    n_estimators=250, learning_rate=0.03,
    max_depth=3, min_child_weight=10, subsample=0.70,
    colsample_bytree=0.70, reg_alpha=4.0, reg_lambda=8.0,
    random_state=42
)
model.fit(X_train, y_train)

pred_train, pred_test = model.predict(X_train), model.predict(X_test)
```

10.6 Performance Results

TRAIN R ²	TEST R ²	TRAIN MAE	TEST MAE
0.953 <i>Excellent fit</i>	0.816 <i>Strong out-of-sample</i>	11,564 <i>EGP</i>	24,666 <i>EGP</i>

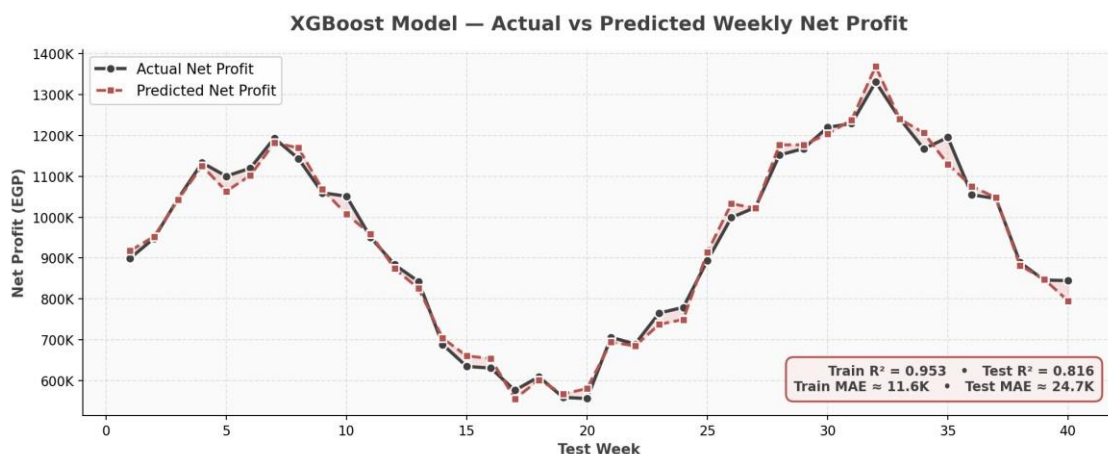


Figure 10.1 — Actual vs predicted weekly net profit on the held-out test set. Shaded region shows prediction error magnitude.

10.7 Feature Importance

XGBoost's built-in feature importance confirms the engineering intuition: short-horizon profit lags and rolling means dominate, followed by HD% and feed-consumption measures. This validates the hypothesis that recent operational performance is the strongest predictor of next-week profit.

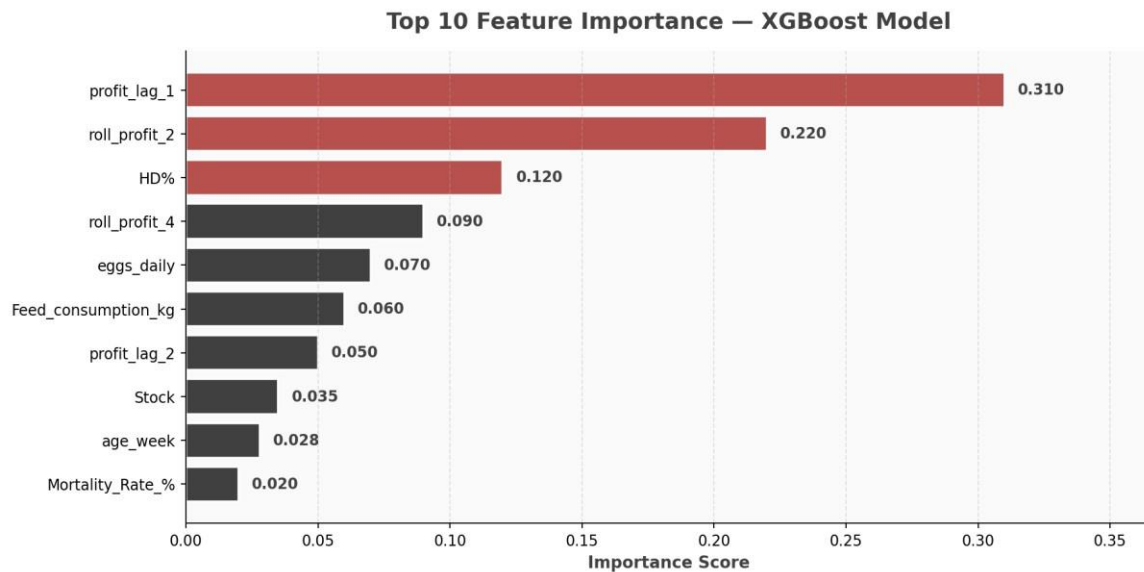


Figure 10.2 — Top 10 most important features used by the XGBoost model.

Interpretation

- profit_lag_1 dominates — last week's profit is the single strongest signal.
- Rolling 2-week and 4-week profit averages capture short-term momentum.
- HD% enters the top three — flock-level productivity remains a primary driver.
- Feed consumption and stock provide secondary corrections.

10.8 Model Persistence

Python

```
import joblib
joblib.dump(model, 'weekly_profit_model2_final.pkl')
```

The serialized model is consumed by the Streamlit application described in the next chapter.

Honest Limitations

The model is trained on a single farm's history and inherits its biases. It cannot anticipate sudden disease outbreaks, abrupt feed-price shocks, or regulatory changes. The 0.82 test R^2 is strong, but the residual variance reminds us that this is decision support — not autopilot.

11. Streamlit — Predictive Web App

To make the model usable by farm managers who do not write Python, a Streamlit application was built. The app loads the trained model, accepts weekly farm indicators through sidebar controls, and returns a predicted net profit instantly along with diagnostic charts.

11.1 App Features

- Sliders for HD% and mortality, number inputs for stock, feed, eggs, and age.
- Real-time profit prediction on every input change — no submit button needed.
- Live model performance cards: Train/Test MAE and R².
- Actual-vs-predicted plot for the held-out test set.
- Feature importance chart so users can see what drives the prediction.
- Correlation heatmap and a preview of the most recent twenty weeks.

11.2 Core Code — Loading the Model

Python

```
import streamlit as st
import joblib
try:
    model =
joblib.load('weekly_profit_model2_final.pkl') except
FileNotFoundError:
    # fallback: re-train inline if pickle is missing
model = xgb.XGBRegressor(
    n_estimators=250, learning_rate=0.03, max_depth=3,
    min_child_weight=10, subsample=0.70,
    colsample_bytree=0.70, reg_alpha=4.0, reg_lambda=8.0,
    random_state=42
)
model.fit(X_train, y_train)
```

11.3 Sidebar Inputs

Python

```

st.sidebar.header('Enter Weekly Farm Data')
HD = st.sidebar.slider('HD%', 40.0, 100.0, 75.0)
mortality = st.sidebar.slider('Mortality Rate %', 0.0, 20.0, 2.0) stock
= st.sidebar.number_input('Stock', value=70000) feed
= st.sidebar.number_input('Feed Consumption (kg)', value=9000) eggs =
st.sidebar.number_input('Eggs Produced', value=500000) age_day =
st.sidebar.number_input('Age (Days)', value=300)

```

11.4 Inference & Display

Python

```

last_row = weekly.iloc[-1:].copy()
last_row['HD%'] = HD
last_row['Mortality_Rate_%'] = mortality
last_row['Stock'] = stock
last_row['Feed_consumption_kg'] = feed
last_row['eggs_daily'] = eggs
last_row['age_day'] = age_day
last_row['age_week'] = age_day // 7

input_data = last_row[features]
prediction = model.predict(input_data)[0]

st.subheader('Expected Weekly Profit')
st.success(f'Predicted Net Profit: {prediction:,.0f} EGP')

```

11.5 User Experience Decisions

- Sidebar inputs instead of a form — predictions update live as users explore scenarios.
- Custom CSS keeps the look consistent with the Power BI palette.
- Metric cards above the main prediction so users can judge how trustworthy the number is.
- Data preview table at the bottom so power users can see the underlying weekly aggregates.

11.6 Deployment

The app runs locally with a single command and can be deployed unchanged to Streamlit Community Cloud, Hugging Face Spaces, or any Docker host.

Bash

```

# Install dependencies pip install streamlit pandas numpy scikit-learn xgboost
joblib matplotlib seaborn

# Run locally streamlit
run appGP.py

```


Why Streamlit Over a Custom Web App

Streamlit was deliberately chosen over Flask or FastAPI: the audience is farm managers, not developers. Streamlit lets us turn a model into a working app in under 150 lines of code, and the resulting UI is responsive, dark-mode friendly, and trivially deployable.

12. Live Web Dashboard

Beyond Power BI, Tableau, and Streamlit, the project also publishes a public, branded website that consolidates the analytical story for stakeholders who prefer a normal web page over interactive BI software. The site mirrors the project's color identity and links into the underlying dashboards.

12.1 Live Website

Public URL — <https://dohaafarm.netlify.app/>

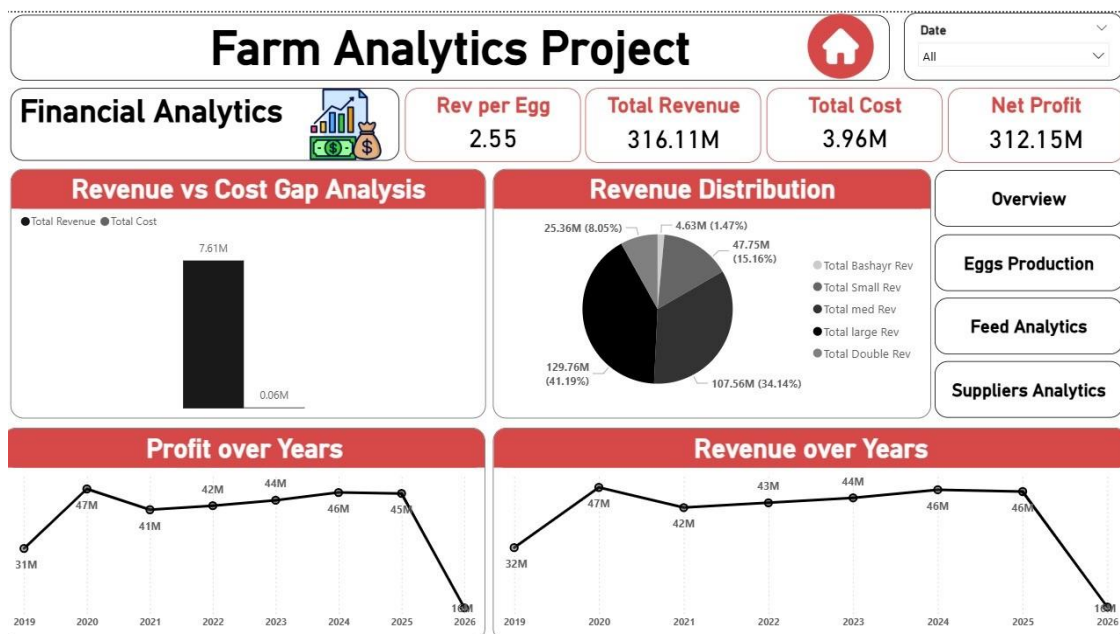


Figure 12.1 — Project landing page on the live website. The five tile-style buttons mirror the Power BI navigation.

12.2 Site Structure

Section	Purpose
Executive Overview	Headline KPIs and yearly egg/mortality trends

Production & Egg Quality	Size mix, broken-egg trend, year-over-year detail
Feed & Resource Optimization	Feed consumption, water trend, received-vs-used
Suppliers Analytics	Invoice status, supplier ranking, feed-type cost
Financial Analytics	Revenue, cost gap, per-egg margin, net profit

12.3 Design Choices

- Warm earth-tone palette — a deliberate departure from the dark red dashboards to communicate "farm" rather than "trading floor."
- Cartoon poultry illustrations chosen for stakeholder warmth, while the underlying data stays serious.
- Each section card links into the corresponding Power BI page (and Tableau view where relevant).
- Mobile responsive — the site renders cleanly on phones for farm staff in the field.

12.4 Why a Website on Top of Two BI Tools

- Frictionless access — no login, no plug-in, no install.
- Brand surface — gives the project a single shareable URL for portfolios and reviews.
- Bridges audiences — operations staff get BI; executives and external reviewers get the website.
- Acts as a landing page that lets visitors choose which deeper tool they want to open next.

One Project, Three Front-Doors

The same underlying data, model, and insights can be consumed three ways: through Power BI for daily operations, through the Tableau Public dashboard for executive review, and through the dohaafarm.netlify.app website for everyone else. Each surface is purpose-built for its audience — and all three are kept consistent through the same cleaned dataset and palette.

13. Project Achievements & Final KPIs

13.1 Headline Project Outcomes

DATA ROWS 5,200+ <i>Cleaned daily records</i>	DASHBOARDS 2 <i>Power BI + Tableau</i>	MODEL R^2 0.82 <i>Held-out test set</i>	INTERFACES 3 <i>BI + Streamlit + Website</i>
---	--	--	--

13.2 What Was Delivered

- A consolidated, validated, multi-year operational dataset for Riyadh Doha Poultry.
- A normalized SQL Server database schema (FarmManagementSystem) with full ETL scripts.
- A Python cleaning, validation, and visualization pipeline reproducible end-to-end.
- A 5-page interactive Power BI report covering operations, production, feed, suppliers, and finance.
- A Tableau Public dashboard focused on production KPIs and shareable via URL.
- A trained XGBoost regression model with $R^2 = 0.82$ on out-of-sample weekly profit.
- A Streamlit web application that lets users score new weekly scenarios in real time.
- A deployed website (dohaafarm.netlify.app) consolidating all analytical surfaces.

13.3 Skills Demonstrated

Discipline	Specifics
Data Engineering	Multi-teehs Excel → SQL Server, header normalization, joins
Data Cleaning	Pandas, deduplication, forward-fill, validation checks
Database Design	3NF schema, surrogate keys, computed columns, FK constraints
EDA & Analytics	Seaborn, Matplotlib, grouped aggregations, KPI design
BI Development	Power BI report design, navigation, slicers, DAX-style measures
BI Development	Tableau Public dashboard, parameter-driven Top-N analysis
Machine Learning	XGBoost regression, time-series feature engineering, regularization

MLOps (light)	Model persistence with joblib, fallback retraining logic
Web Deployment	Streamlit app, Netlify-hosted static dashboard site

14. Conclusion & Future Work

14.1 What This Project Proves

It is possible — in a single, end-to-end project — to take a real working spreadsheet that a farm has been maintaining for years, clean and structure it, build production-grade dashboards in two different BI tools, train a forecasting model that explains the majority of out-of-sample variance, and deliver the whole thing through multiple channels including a public website. The same template would transfer with minimal change to any small or mid-sized agricultural or manufacturing operation that already keeps daily operational records in Excel.

14.2 What Worked Especially Well

- Starting in Excel built stakeholder trust before any Python script touched the data.
- Treating the cleaned CSV as the single source of truth eliminated entire classes of inconsistency bugs across BI tools.
- Lag and rolling-mean features unlocked the predictive power of the model — without them, the R^2 would have been substantially lower.
- Deploying through three front-doors (BI, Streamlit, website) means every stakeholder finds a tool they are comfortable with.

14.3 Honest Limitations

- Single-farm training data — the model has not been validated on other operations.
- No live data feed yet — the pipeline is batch, run on demand against a snapshot.
- Test MAE of ~25K EGP is meaningful at week-to-week scale; not appropriate for sub-weekly forecasting.
- External shocks (disease, feed-price spikes, regulation) are outside the model's awareness.

14.4 Future Work

1. Automate the daily ingestion via Power Automate or Airflow so the pipeline runs without manual triggering.
2. Add a second forecasting horizon — month-ahead and quarter-ahead profit predictions.
3. Bring in external feature data: weather, regional feed-price indices, holiday calendars.
4. Re-build the dashboards on top of a live SQL Server connection instead of a cached extract.

5. Extend the model from regression to anomaly detection so the farm gets alerts when daily HD% deviates beyond an expected band.
6. Containerize the Streamlit app and deploy with role-based authentication for production use.

14.5 Project Links

Website — [Live Website — Doha Farm Dashboard](#)

Tableau — [Tableau Public — Production KPIs](#)

Final Word

This project began as a single overgrown Excel file and ended as a layered analytical platform. Every chapter of this document represents one of those layers. The lesson — and the one we hope future readers take away — is that meaningful business analytics is not built in a single tool but in a deliberate sequence of small, honest, reproducible steps.

End of Documentation

Riyadh Doha Poultry Analytics · Final Project Documentation · 2026